

Supplementary material for *couplr: Optimal Matching with Verifiable Certificates and Sparse Edge Generation*

Gilles Colling

Contents

1	What this file holds	1
2	The sentinel padding lemma	2
3	Solver glossary	3
4	From matched pairs to an estimate	4
5	Solver selection across problem regimes	5
5.1	What the rules cost	6
5.2	Every solver against the fastest in its cell	7
5.3	Do the solvers agree	7
6	The edge-generation loop away from one cloud	8
7	Peak memory	9
8	Representation against solver	11
9	Scaling	11
10	Software design	12
10.1	Object model	12
10.2	C++ organisation	12
10.3	Dependency footprint	13

1 What this file holds

The article reports summaries of four measurements that are too large to print in full: the solver grid over problem regimes, the edge-generation loop over covariate counts, point clouds, calipers and sizes, the peak memory of every mode and comparator, and the raw runs behind the scaling table. This file carries the definitions the measurements are stated over, the protocol they were taken under, and summary and worst-case views of each one, and names the script that produced each so any of them can be re-measured. It also carries the parts of the software design the article points at: the object model, the C++ organisation and the dependency footprint, the proof of the sentinel-padding lemma the article states, the glossary behind the solver figure’s labels, and the outcome analysis the article points at.

Every cell of every measurement is in the CSV files listed below, which ship with the submission under `data/`. The tables here select from those files; they do not replace them. Each section names the file that carries its cells in full.

```
## run      20260830-152628
## host     Darwin Mac 25.5.0 Darwin Kernel Version 25.5.0: Mon Apr 27 20:41:15
##          PDT 2026; root:xnu-12377.121.6~2/RELEASE_ARM64_T6041 arm64
## commit   a63b760 28 modified paths
## R version 4.5.3 (2026-03-11)
```

```

## aarch64-apple-darwin25.3.0
## couplr          1.7.0
## MatchIt        4.7.2
## optmatch       0.10.8
## microbenchmark 1.5.0
##
## note           Every arm the article reports was measured at this one run and
##                commit. The harness read 1.6.2 from the DESCRIPTION at a63b760 and
##                the version is recorded here as 1.7.0: no commit between a63b760 and
##                the 1.7.0 release touches a file under R/ or src/, so the sources
##                that produced these numbers are the released ones and only the
##                version string differs. The modified paths the run recorded are the
##                benchmark outputs it was writing, together with two RcppExports
##                files that differ from the commit only in line endings.

```

Section	Script	Files
Solver selection across regimes	scripts/bench_regimes.R	regime-runs.csv, regime-cells.csv, regime-results.csv, regime-verdict.csv
The edge-generation loop	scripts/bench_implicit_grid.R	implicit-grid-runs.csv, implicit-grid-results.csv
Peak memory	scripts/bench_memory.R	memory-results.csv
Representation against solver	scripts/bench_implicit.R	implicit-results.csv
Scaling	scripts/bench_scaling.R	scaling-runs.csv, scaling-results.csv
The solver figure	scripts/make-figure.R	benchmark-table.csv

Every script is resume-safe: it writes its rows as it goes and skips a cell it already holds, so a run that is interrupted continues where it stopped. Moving the CSV aside forces a full re-measurement. `paper/run_bench_suite.sh` runs all of them in order on one machine, which is how the numbers here were produced.

The scripts load couplr from a source checkout with `pkgload::load_all()` and write their CSVs under `paper/`, so they run from a clone of the package repository rather than from the submission archive:

```

git clone https://github.com/gcol33/couplr
cd couplr
sh paper/run_bench_suite.sh

```

The scaling, implicit, implicit-grid, regimes and memory scripts also take `--quick`, which runs a reduced grid.

2 The sentinel padding lemma

Tight constraints can leave the matching problem infeasible, and couplr answers it by replacing the forbidden entries with a finite sentinel and re-solving. The article states the lemma that makes a minimum-cost solve of the padded problem return the lexicographic optimum, the largest admissible matching and the cheapest among those of its size, and defers the proof here.

Write $[\ell, h]$ for the range of the admissible costs of the pruned submatrix, k for the smaller of its two dimensions and σ for the sentinel. Every feasible assignment of the padded problem matches all k rows, so it splits into s sentinel edges and $k - s$ real ones and costs $s\sigma + R$ with $R \in [(k - s)\ell, (k - s)h]$.

Lemma 1. *If $\sigma > k(|\ell| + |h|)$, then a padded assignment using more sentinel edges than another costs strictly more.*

Proof. Let M' and M use s' and s sentinel edges with $s' > s$, and write R' and R for their real costs.

Then

$$\text{cost}(M') - \text{cost}(M) = (s' - s)\sigma + R' - R \geq \sigma + (k - s')\ell - (k - s)h \geq \sigma - k|\ell| - k|h| > 0. \quad \square$$

couplr takes $\sigma = (k + 1)(|\ell| + |h|) + 1$, which satisfies the hypothesis whatever the signs of ℓ and h and when the admissible costs are all zero, and leaves a margin of $|\ell| + |h| + 1$ between a matching and one using more sentinel edges. `assignment(cardinality = "maximum")` prices its dummy columns with the same quantity, and a maximisation is solved by negation with the sentinel negated alongside it, so the lemma applies to the negated instance. The reported objective is recomputed over the real pairs, so no sentinel enters it.

The lemma’s hypothesis is a statement about the numbers actually used, so it is checkable on the doubles in hand and couplr’s σ satisfies it by construction. What the lemma does not settle is the solve. A solver working where a double no longer resolves a gap of $|\ell| + |h| + 1$ cannot be relied on to order two assignments the padding separates by that much, so couplr refuses a padded objective above 2^{53} rather than returning an ordering its arithmetic may not support. That threshold is a precondition on the solve and not a proof of it: 2^{53} bounds the integers a double carries exactly, and the padded costs need not be integers. What establishes that a particular padded solve reached its optimum is the certificate `verify_assignment()` returns for the padded matrix, and the lemma carries that verdict across to the lexicographic objective.

3 Solver glossary

The article’s solver figure labels its traces with abbreviations short enough to sit inside a panel. The mapping below gives, for each label, the value the `method` argument of `assignment()` and `lap_solve()` takes, the family the benchmark table groups it under, and the regime the solver is written for. The labels and families are read from `benchmark-table.csv`, the same file the figure is drawn from.

Table 1: Plot label, argument value, family and intended regime for every solver the article’s figure draws.

Label	method =	Family	Meaning
auto	auto	Auto	The dispatch rules, timed as a solver of its own so the probing pass is inside its number.
Auction	auction	Auction	Bertsekas auction with adaptive epsilon.
Auction-GS	auction_gs	Auction	Gauss-Seidel auction variant; spatial structure.
Auction-S	auction_scaled	Auction	Epsilon-scaling auction; large dense problems.
Hungarian	hungarian	Classical	The classical Hungarian method, $O(n^3)$.
Munkres	munkres	Classical	Matrix-form Kuhn-Munkres, $O(n^4)$, kept as a reference implementation.
CSA	csa	Cost scaling	Goldberg-Kennedy cost scaling; medium to large problems.
Gabow-T	gabow_tarjan	Cost scaling	Gabow-Tarjan bit scaling with complementary slackness; integer costs.
SSAP-B	ssap_bucket	Cost scaling	Dial bucket priority queue over augmenting paths; integer costs.
CS-flow	csflow	Flow-based	Successive shortest paths with Johnson potentials on the flow model.
Cycle-C	cycle_cancel	Flow-based	Cycle cancelling with Karp’s minimum-mean cycle.
Net-S	network_simplex	Flow-based	Network simplex with a spanning-tree representation.
Push-R	push_relabel	Flow-based	Goldberg-Tarjan cost-scaling push-relabel.
JV	jv	JV / Augmenting path	Column reduction and auction-style initialisation; the general-purpose default on a dense problem.

Label	method =	Family	Meaning
LAPMOD	lapmod	JV / Augmenting path	Sparse Jonker-Volgenant variant over an adjacency list; more than half the entries forbidden.
SAP	sap	JV / Augmenting path	Shortest augmenting path over the shared flow model; carries sparsity well.
SAP-D	sap_dense	JV / Augmenting path	Shortest augmenting path with a linear scan in place of a heap; a dense cost matrix.
Ramshaw	ramshaw_tarjan	Rectangular / general	Ramshaw-Tarjan; strongly rectangular problems, where padding to a square would dominate.
Brute-F	bruteforce	Special-purpose	Exact enumeration; at most eight rows and eight columns.
HK-01	hk01	Special-purpose	Hopcroft-Karp cardinality matching; binary or constant finite costs.

Three further routines are reachable but not drawn in the figure: the bottleneck objective, which minimises the largest pair cost instead of the sum, Murty's ranking procedure for the k cheapest assignments, and Sinkhorn iteration for the entropy-regularised relaxation.

4 From matched pairs to an estimate

The article shows the matched sample and defers the outcome analysis here. The match is rebuilt below so this file stands on its own.

```
library(couplr)
data(hospital_staff)
treated <- transform(hospital_staff$nurses_extended, id = nurse_id)
control <- transform(hospital_staff$controls_extended, id = nurse_id)
covars <- c("age", "experience_years", "certification_level")
m <- match_couples(treated, control, vars = covars, auto_scale = TRUE)
```

Taking the hourly rate as the outcome, the paired difference within each matched pair estimates the association between full-time status and pay among nurses comparable on age, experience and certification.

```
matched <- join_matched(m, treated, control)
d <- matched$hourly_rate_left - matched$hourly_rate_right
tt <- t.test(d)
c(estimate = mean(d), lower = tt$conf.int[1], upper = tt$conf.int[2])
```

```
## estimate    lower    upper
## 3.531350 1.875651 5.187049
```

Full-time nurses earn 3.53 more per hour than their matched part-time counterparts, with a 95 percent interval that excludes zero. As a paired difference on a matched sample this estimates an adjusted association. Reading it as the causal effect of full-time status is what rests on assumptions no matching method can verify: conditional ignorability, positivity, and the stable unit treatment value assumption (SUTVA), under which a unit's outcome depends on its own treatment alone. The matched output can also be passed to an outcome model for regression adjustment; valid estimation and uncertainty quantification still depend on the estimator used.

What matching can offer against the first of those assumptions is a statement about how strong an unmeasured confounder would have to be to overturn the result. `sensitivity_analysis()` implements Rosenbaum bounds (Rosenbaum 2002) for the Wilcoxon signed-rank statistic on one-to-one matched pairs, reporting upper and lower p-value bounds across a grid of the odds-ratio parameter Γ .

```
sens <- sensitivity_analysis(m, treated, control, outcome_var = "hourly_rate")
sens$results[sens$results$gamma %in% c(1, 1.5, 1.75, 2), ]
```

```
## # A tibble: 4 x 4
##   gamma t_stat  p_upper p_lower
##   <dbl> <dbl>    <dbl>  <dbl>
## 1 1     13304. 0.0000187 1.87e- 5
## 2 1.5   13304. 0.0435     1.01e-11
## 3 1.75 13304. 0.207     4.48e-15
## 4 2     13304. 0.481     1.69e-18
```

At $\Gamma = 1$, meaning no hidden bias, the association is significant. It survives an unmeasured confounder that raises one pair member's odds of being full-time by half, and stops being significant at $\Gamma = 1.75$. A reader can judge from that number whether a confounder of that strength is plausible in this setting; the analysis quantifies exposure to hidden bias rather than removing it.

5 Solver selection across problem regimes

`assignment(method = "auto")` chooses a solver from five rules read off one pass over the cost matrix: enumerate an at most 8 by 8 problem, use a cardinality algorithm where the finite costs are constant or binary, use the sparse Jonker-Volgenant variant where more than half the entries are forbidden, use the shortest-augmenting-path solver where there are at least three times as many columns as rows, and otherwise use Jonker-Volgenant. The article's solver figure varies problem size on square dense integer costs, which is the last rule's regime and no other. This grid varies the properties the rules are stated over.

A cell is a cost regime, a forbidden pattern and a shape. Each cell is generated as several independent instances, each instance is timed several times, and every solver in the panel sees the same instance. The timing a cell reports for a solver is the median across its instances, where each instance contributes the median of its own repetitions, so the spread is between problems rather than between repetitions of one problem. A solver that errored or timed out on a cell has no row there.

Table 2: The factors the solver grid crosses. The admissible shares are the medians of the shares the generated cells actually held. Shapes and sizes are in the next table.

Factor	Level	Meaning
cost regime	int_uniform	integer, uniform on 1..10,000
cost regime	dbl_uniform	double, uniform on (0, 1)
cost regime	binary	binary, 0 or 1
cost regime	constant	constant
cost regime	tied	integer, five distinct values
cost regime	heavy_tailed	double, lognormal
cost regime	metric_uniform	euclidean over uniform points in 8 dimensions
cost regime	metric_clustered	euclidean over eight clusters in 8 dimensions
forbidden pattern	block_4	25 percent of pairs admissible, in 4 disconnected components
forbidden pattern	none	every pair admissible, unstructured
forbidden pattern	random_01	1 percent of pairs admissible, unstructured
forbidden pattern	random_05	5 percent of pairs admissible, unstructured
forbidden pattern	random_25	25 percent of pairs admissible, unstructured
forbidden pattern	random_60	60 percent of pairs admissible, unstructured

Table 3: Shapes. The base tier runs the whole panel at a size every solver in it reaches; the large tier repeats the crossing at a size only the scalable solvers reach.

tier	Shape	aspect	Solvers in the panel
base	500x500	1:1	12
base	500x1500	1:3	12

tier	Shape	aspect	Solvers in the panel
base	500x5000	1:10	12
large	1000x10000	1:10	5
large	1500x1500	1:1	5
large	1500x4500	1:3	5

5.1 What the rules cost

For each cell, the solver "auto" selected is timed as a solver of its own, so its number carries the probing pass the rules cost, and it is compared against the fastest named solver in the same cell. Ratio below is the time the dispatched solver took over the time the cell's fastest named solver took, so 1.00x means the rule picked the best available solver and 2.00x means it cost twice the best.

Table 4: Each dispatch rule, over the cells where it fired.

Rule	Cells	Picked the cell's fastest	Median ratio	Worst ratio
default	144	59	1.04x	9.95x
no_cost_scale	45	35	1.00x	1.93x

Table 5: The same comparison collapsed over each factor of the grid in turn.

Factor	Level	Cells	Picked the cell's fastest	Median ratio	Worst ratio
cost regime	binary	27	17	0.99x	1.93x
cost regime	constant	18	18	1.02x	1.38x
cost regime	dbl_uniform	27	13	1.02x	3.67x
cost regime	heavy_tailed	27	13	1.02x	3.23x
cost regime	int_uniform	27	13	1.02x	3.89x
cost regime	metric_clustered	27	14	1.04x	3.56x
cost regime	metric_uniform	18	5	1.02x	3.51x
cost regime	tied	18	1	3.98x	9.95x
forbidden pattern	25%, 4 blocks	24	11	1.04x	4.32x
forbidden pattern	complete	39	27	1.00x	8.87x
forbidden pattern	1%	39	14	1.11x	3.89x
forbidden pattern	5%	24	7	1.23x	9.91x
forbidden pattern	25%	39	23	1.04x	9.95x
forbidden pattern	60%	24	12	1.02x	7.82x
tier	base	144	64	1.04x	9.95x
tier	large	45	30	1.00x	3.18x

Table 6: The 15 cells where the dispatched solver was furthest off the cell's fastest.

Regime	Pattern	Shape	Picked	Best	Auto (ms)	Best (ms)	Ratio
tied	25%	500x500	jv	auction_scaled	63.6	6.4	9.95x
tied	5%	500x500	jv	auction_scaled	41.5	4.2	9.91x
tied	complete	500x500	jv	network_simplex	57.9	6.5	8.87x
tied	complete	500x1500	jv	network_simplex	204	23.4	8.72x
tied	60%	500x500	jv	network_simplex	63.9	8.2	7.82x
tied	25%	500x1500	jv	network_simplex	203	37.2	5.45x
tied	60%	500x1500	jv	network_simplex	209	38.6	5.41x
tied	complete	500x5000	jv	network_simplex	700	132	5.29x
tied	25%, 4 blocks	500x500	jv	auction_scaled	17.9	4.1	4.32x

Regime	Pattern	Shape	Picked	Best	Auto (ms)	Best (ms)	Ratio
int_uniform	1%	500x500	jv	auction_scaled	5.5	1.4	3.89x
dbl_uniform	1%	500x500	jv	auction_scaled	6.4	1.7	3.67x
tied	60%	500x5000	jv	network_simplex	716	197	3.64x
metric_clustered	5%	500x500	jv	auction_scaled	9	2.5	3.56x
metric_uniform	5%	500x500	jv	auction_scaled	8.1	2.3	3.51x
metric_clustered	1%	500x500	jv	csa	5.4	1.6	3.46x

The rule, the solver it picked, the fastest solver and the ratio between them are in `regime-verdict.csv` for all 189 cells.

5.2 Every solver against the fastest in its cell

The table below reduces each solver to its position across the grid. A solver’s ratio in a cell is its median time over the median time of the fastest solver in that cell, so the reduction is within-cell and a solver that entered only the easy cells is not thereby favoured. Solvers differ in how many cells they entered, since the large tier runs a reduced panel and a solver that errored or timed out on a cell has no row there.

Table 7: Every solver over the cells it entered, as a ratio to the fastest solver in the same cell.

Solver	Cells entered	Times fastest	Median ratio	Q75 ratio	Worst ratio	Fastest cell (ms)	Slowest cell (ms)
hk01	36	25	1.00x	1.00x	2.1x	1.15	43.7
auto	189	47	1.02x	1.33x	10.0x	1.29	716
jv	189	35	1.22x	5.48x	84.3x	3.19	5.5e+03
hungarian	144	27	1.61x	8.73x	53.6x	3.71	716
lapmod	189	3	1.72x	7.67x	93.9x	4.1	7.95e+03
csflow	144	11	2.70x	6.97x	56.6x	2.82	879
sap	189	5	2.70x	6.57x	71.9x	2.68	6.43e+03
sap_dense	144	0	2.85x	14.46x	86.3x	6.76	2.15e+03
ramshaw_tarjan	189	0	2.88x	8.60x	107.5x	5.23	1.05e+04
network_simplex	144	8	17.01x	26.73x	70.2x	3.09	793
csa	144	2	31.30x	131.64x	1108.2x	1.55	1.34e+04
auction_scaled	137	26	35.14x	127.81x	670.3x	1.42	8.21e+03

The per-cell medians and quartiles behind this table are in `regime-results.csv`, 1838 rows covering 189 cells and 12 solvers, and the individual timed repetitions are in `regime-runs.csv`, 15428 rows.

5.3 Do the solvers agree

Every solver in a cell solves the same instance, so their totals have to agree. Each run below is compared against the median total of its own cell, and a run counts as off that median where the difference exceeds 10^{-6} of it.

522 instances carry 5 to 12 solvers each. 10 of the 12 solvers return the cell median exactly on every run. The other 2 are below.

Table 8: The solvers whose total differed from the median total of a cell they ran in.

Solver	Runs	Runs off the median	Regimes	Worst relative difference
csa	1296	159	heavy_tailed	696
auction_scaled	1218	9	heavy_tailed	1.23e-05

Both are scaling methods, and both deviate on the lognormal regime alone, so the rounding a scaling step applies over a cost range that wide may be what separates them from the rest. The size of the two deviations differs by orders of magnitude, and neither is a property of the regime: the remaining solvers return one common total on every cell of it. Whether a particular matching is optimal for the matrix as supplied is what `verify_assignment()` answers.

6 The edge-generation loop away from one cloud

The article’s implicit results come from one cloud: eight independent normal covariates with the treated group shifted, one draw per size. This grid sweeps one factor at a time around that base, several seeds deep. Where the dense path can afford the problem, the loop’s answer is compared against it unit by unit.

Table 9: The loop by cell. A sweep moves one factor and holds the rest at the base cell, a gaussian cloud of 10,000 units in 8 dimensions with no caliper. Rounds is the range over seeds. Graph is the median share of the complete pair count the candidate set ended up holding, in percent. Distances is the median number of distance evaluations over all rounds as a multiple of the complete pair count. Implicit and lazy are median seconds over seeds and speedup is their median ratio.

Sweep	Level	Seeds	Rounds	Graph %	Distances	Implicit	Lazy	Speedup
caliper	boundary	5	2-2	1.151	0.31x	0.91 s	1.66 s	1.79x
caliper	mid	5	2-2	1.163	0.51x	0.989 s	1.8 s	1.82x
caliper	none	5	2-2	1.170	1.79x	0.863 s	1.77 s	2.05x
cloud	clustered	5	2-2	1.170	0.48x	0.324 s	0.654 s	2.02x
cloud	contested	5	9-10	73.318	6.67x	134 s	426 s	3.17x
cloud	gaussian	5	2-2	1.170	1.79x	0.864 s	1.76 s	2.05x
cloud	heavy_tailed	5	2-2	1.170	1.99x	0.894 s	0.967 s	1.09x
cloud	lattice_ties	5	2-2	1.170	1.78x	1.09 s	1.96 s	1.79x
cloud	shell	5	4-7	1.183	4.96x	2.63 s	14.8 s	5.57x
cloud	shifted	5	7-7	3.086	5.92x	4.81 s	54.3 s	11.13x
dimension	2 dimensions	5	2-4	1.170	0.24x	0.167 s	0.599 s	3.76x
dimension	4 dimensions	5	2-2	1.170	0.69x	0.336 s	1.45 s	4.36x
dimension	8 dimensions	5	2-2	1.170	1.79x	0.864 s	1.77 s	2.05x
dimension	16 dimensions	5	2-2	1.170	2.00x	2.51 s	4.96 s	1.97x
dimension	32 dimensions	5	2-2	1.170	2.00x	9.57 s	21.3 s	2.22x
size	2,000 units	3	2-2	4.951	1.99x	0.05 s	0.065 s	1.27x
size	5,000 units	3	2-2	2.160	1.92x	0.25 s	0.369 s	1.48x
size	10,000 units	3	2-2	1.170	1.79x	0.86 s	1.76 s	2.05x
size	20,000 units	3	2-2	0.630	1.61x	2.97 s	7.67 s	2.56x
size	50,000 units	3	2-3	0.288	1.94x	21.7 s	64.3 s	3.02x

Table 10: The certificate by cell. The duality gap is the residual the loop stopped on. Equals the dense solve compares the pairing unit by unit where the dense path could be run, and reads not run where the problem was too large for it.

Sweep	Level	Worst duality gap	All certified	Equals the dense solve
caliper	boundary	0.00e+00	TRUE	TRUE
caliper	mid	0.00e+00	TRUE	TRUE
caliper	none	0.00e+00	TRUE	TRUE
cloud	clustered	0.00e+00	TRUE	TRUE
cloud	contested	0.00e+00	TRUE	TRUE
cloud	gaussian	0.00e+00	TRUE	TRUE

Sweep	Level	Worst duality gap	All certified	Equals the dense solve
cloud	heavy_tailed	0.00e+00	TRUE	TRUE
cloud	lattice_ties	0.00e+00	TRUE	TRUE
cloud	shell	0.00e+00	TRUE	TRUE
cloud	shifted	0.00e+00	TRUE	TRUE
dimension	2 dimensions	0.00e+00	TRUE	TRUE
dimension	4 dimensions	0.00e+00	TRUE	TRUE
dimension	8 dimensions	0.00e+00	TRUE	TRUE
dimension	16 dimensions	0.00e+00	TRUE	TRUE
dimension	32 dimensions	0.00e+00	TRUE	TRUE
size	2,000 units	0.00e+00	TRUE	TRUE
size	5,000 units	0.00e+00	TRUE	TRUE
size	10,000 units	0.00e+00	TRUE	TRUE
size	20,000 units	0.00e+00	TRUE	not run
size	50,000 units	3.64e-12	TRUE	not run

The medians above are taken over seeds, and the seed-to-seed spread is what decides whether they carry. The next table reports, for each cell, the widest separation the seeds produced.

Table 11: Across the seeds of a cell: the range of each quantity and the ratio of the slowest run to the fastest. Seed width is the number of columns the first round gave each row.

Sweep	Level	Seeds	Rounds	Seed width	Graph %	Seconds	Slowest / fastest
caliper	boundary	5	2-2	78-78	1.121-1.168	0.841-1.05	1.25x
caliper	mid	5	2-2	78-78	1.125-1.167	0.85-1.03	1.22x
caliper	none	5	2-2	78-78	1.170-1.170	0.861-0.866	1.01x
cloud	clustered	5	2-2	78-78	1.170-1.170	0.313-0.36	1.15x
cloud	contested	5	9-10	78-78	73.287-73.355	133-135	1.01x
cloud	gaussian	5	2-2	78-78	1.170-1.170	0.862-0.868	1.01x
cloud	heavy_tailed	5	2-2	78-78	1.170-1.170	0.891-0.914	1.03x
cloud	lattice_ties	5	2-2	78-78	1.170-1.170	1.09-1.1	1.00x
cloud	shell	5	4-7	78-78	1.172-2.825	1.95-3.4	1.75x
cloud	shifted	5	7-7	78-78	3.062-3.108	4.72-4.92	1.04x
dimension	2 dimensions	5	2-4	78-78	1.170-1.170	0.14-0.183	1.31x
dimension	4 dimensions	5	2-2	78-78	1.170-1.170	0.332-0.34	1.02x
dimension	8 dimensions	5	2-2	78-78	1.170-1.170	0.862-0.869	1.01x
dimension	16 dimensions	5	2-2	78-78	1.170-1.170	2.51-2.51	1.00x
dimension	32 dimensions	5	2-2	78-78	1.170-1.170	9.57-9.57	1.00x
size	2,000 units	3	2-2	66-66	4.951-4.951	0.05-0.051	1.02x
size	5,000 units	3	2-2	72-72	2.160-2.160	0.249-0.251	1.01x
size	10,000 units	3	2-2	78-78	1.170-1.170	0.857-0.866	1.01x
size	20,000 units	3	2-2	84-84	0.630-0.630	2.96-2.99	1.01x
size	50,000 units	3	2-3	96-96	0.288-0.288	15.2-21.8	1.44x

Every cell of the grid finished: 264 runs over 20 cells and 3 memory modes, all with status ok.

Each of the 264 runs behind these tables is a row of `implicit-grid-runs.csv`, carrying its seed, mode, timing, round count, candidate and evaluated edge counts, duality gap and comparison against the dense solve. The per-cell medians are in `implicit-grid-results.csv`.

7 Peak memory

Peak resident set size is a property of a process, so each cell here is a fresh R process that loads what it needs, does one matching and exits, measured from outside by the platform's `time` utility. The baseline

row of a size is a process that loaded the same packages and matched nothing, and it is what the increment column is measured against. `dense_matrix` builds the distance matrix and stops, which separates the representation from the solver that runs on it.

Table 12: Peak resident set size by arm and problem size, in megabytes.

Units	Arm	Peak RSS	Over baseline	R heap peak	Seconds	Outcome
2,000	baseline	146	-	80	0.007	ok
2,000	baseline_alt	153	-	87	0.008	ok
2,000	dense	265	119	153	0.167	ok
2,000	dense_matrix	203	56	137	0.071	ok
2,000	implicit	158	12	102	0.071	ok
2,000	lazy	151	5	102	0.068	ok
2,000	MatchIt	530	377	311	2.38	ok
2,000	optmatch	480	327	318	1.89	ok
5,000	baseline	146	-	80	0.008	ok
5,000	baseline_alt	153	-	87	0.008	ok
5,000	dense	566	420	409	0.902	ok
5,000	dense_matrix	226	80	161	0.343	ok
5,000	implicit	175	30	104	0.313	ok
5,000	lazy	154	8	104	0.363	ok
5,000	MatchIt	2436	2283	1474	16.8	ok
5,000	optmatch	2059	1907	1320	13.4	ok
10,000	baseline	146	-	80	0.008	ok
10,000	baseline_alt	154	-	87	0.008	ok
10,000	dense	1429	1283	812	3.19	ok
10,000	dense_matrix	446	300	374	1.23	ok
10,000	implicit	205	59	108	1.11	ok
10,000	lazy	159	14	108	1.6	ok
10,000	MatchIt	8866	8713	6382	74.6	ok
10,000	optmatch	7333	7179	4545	60.9	ok
20,000	baseline	146	-	80	0.007	ok
20,000	baseline_alt	153	-	87	0.008	ok
20,000	dense	6380	6234	3656	10.7	ok
20,000	dense_matrix	1241	1095	1145	4.3	ok
20,000	implicit	274	128	116	3.92	ok
20,000	lazy	168	22	116	6.42	ok
20,000	MatchIt	8794	8640	7121	114	error
20,000	optmatch	27581	27428	19331	281	ok

Table 13: The four-fold estimate against the measurement, in megabytes. Matrix is the bytes the dense cost matrix itself needs at that shape, and package estimate is what `estimate_dense_matrix_mb()` predicts, which is four times the matrix and is what the memory dispatcher warns from. The estimate is deliberately conservative: it exists to avoid under-warning before a large allocation, not to predict the peak.

Units	Matrix	Measured over baseline	Measured / matrix	Package estimate
2,000	7	56	7.96	28
5,000	44	80	1.81	178
10,000	178	300	1.69	711
20,000	711	1095	1.54	2844

The MatchIt arm at 20,000 units did not finish, with error: (from optmatch) result would exceed

2³¹-1 bytes. All 32 measured processes are in `memory-results.csv`, which also carries the matrix and estimate columns for every arm.

8 Representation against solver

The implicit mode is a representation and a solver at once: its restricted master is the flow model, and naming a solver under it is an error rather than a setting. Reading its timing as a representation comparison would therefore attribute the flow solver’s behaviour to the representation. The table below is the decomposition: down a column, one solver over two representations; across a row, one representation under two solvers.

Table 14: Wall-clock seconds by representation and solver. Dense and lazy differ only in whether the matrix is materialised; dense and dense-flow differ only in the solver; implicit is the mode as a user meets it.

Units	Dense, JV	Lazy, JV	Dense, flow	Implicit
500	22 ms	8 ms	21 ms	12 ms
2,000	161 ms	53 ms	196 ms	56 ms
5,000	837 ms	351 ms	1.14 s	299 ms
10,000	3.14 s	1.59 s	4.17 s	1.09 s
20,000	not run	6.38 s	not run	3.9 s
50,000	not run	62 s	not run	20.2 s

The certificate, pair count, candidate edge count and round count of each of these runs are in `implicit-results.csv`.

9 Scaling

The scaling table in the article reports the median across instances. The dispersion behind those medians is below: an instance is an independently generated problem of that size, a repetition is one timed solve of that problem, and every package sees the same instances.

Table 15: Every size and package of the scaling benchmark. Max over min is the spread across all repetitions of that cell, so it carries both the between-instance and the within-instance variation.

Units	Package	Instances	Runs	Median (s)	Min (s)	Max (s)	Max / min	Outcome
500	couplr	5	15	0.017	0.016	0.021	1.31x	ok
500	MatchIt	5	15	0.134	0.131	0.166	1.27x	ok
500	optmatch	5	15	0.12	0.108	0.153	1.42x	ok
2,000	couplr	5	15	0.14	0.137	0.163	1.19x	ok
2,000	MatchIt	5	15	2.137	2.037	2.283	1.12x	ok
2,000	optmatch	5	15	1.751	1.617	1.794	1.11x	ok
5,000	couplr	5	10	0.789	0.752	0.834	1.11x	ok
5,000	MatchIt	5	10	16.26	15.89	16.67	1.05x	ok
5,000	optmatch	5	10	12.77	12.46	13.18	1.06x	ok
10,000	couplr	3	6	3.158	3.037	3.284	1.08x	ok
10,000	MatchIt	3	6	74.17	72.78	74.52	1.02x	ok
10,000	optmatch	3	6	60.46	59.18	60.56	1.02x	ok
20,000	couplr	2	2	11.62	10.7	12.54	1.17x	ok
20,000	MatchIt	0	0	-	-	-	-	error
20,000	optmatch	2	2	286.7	280.6	292.8	1.04x	ok
50,000	couplr	1	1	76.25	76.25	76.25	1.00x	ok
50,000	MatchIt	0	0	-	-	-	-	timeout

Units	Package	Instances	Runs	Median (s)	Min (s)	Max (s)	Max / min	Outcome
50,000	optmatch	0	0	-	-	-	-	timeout

Table 16: The runs that did not return a time.

Units	Package	Runs	Outcome
20,000	MatchIt	2	error: (from optmatch) result would exceed $2^{31}-1$ bytes
50,000	MatchIt	1	timeout
50,000	optmatch	1	timeout

Each of the 147 individual repetitions, with its instance index, seed, seconds and status, is a row of `scaling-runs.csv`. The per-cell quartiles are in `scaling-results.csv`.

10 Software design

The article describes the three layers the package is organised in, the dispatch rules, the memory modes and how the package is verified. This section carries the object model, the C++ organisation and the dependency footprint.

10.1 Object model

The object model is S3 throughout. Matching results carry the class vector `c("matching_result", "couplr_result")`, with the design-specific classes (`full_matching_result`, `cem_result`, `subclass_result`) sharing the `couplr_result` parent, which is what lets `balance_diagnostics()`, `join_matched()`, `match_data()` and the `bal.tab()` methods dispatch on it once rather than being rewritten per design. Results are plain lists of tibbles, so a user who wants something the package does not provide can reach in and take it. `lap_solve()` returns a tibble subclassed as `lap_solve_result` with `source`, `target` and `cost` columns, so an optimum is usable in a plot or a join without an extraction step, while `get_total_cost()` and `get_method_used()` recover the metadata carried in attributes.

10.2 C++ organisation

All nineteen solvers are written from scratch in C++ and exposed through Rcpp, with `src/core` holding the cost-source types and shared utilities with no Rcpp dependency, `src/solvers` one algorithm per file with a thin `_rcpp.cpp` wrapper each, `src/flow` the flow model, its two solvers and the edge-generation loop, and `src/interface` the conversion between R objects and the internal types. The separation means the solver bodies are ordinary C++ that can be compiled and tested outside R, which is what the C++ test suite does.

Solvers are templated on their cost source rather than written against a concrete matrix. A cost source provides `at(i, j)`, `allowed(i, j)` and `empty()`, and two types satisfy that interface: `CostMatrix`, which owns a dense flat array plus a mask, and `LazyCostMatrix`, which owns the two feature matrices and computes C_{ij} on demand from the requested metric, applying calipers and the distance ceiling in `allowed()`. A handful of hot loops index the dense mask array directly for speed, which the lazy type cannot support; these are guarded by a `supports_raw_mask` trait and an `if constexpr` branch rather than by duplicating the solver. One solver body therefore serves both cost representations, and the pricing oracle of the edge-generation loop needs `at(i, j)` and `allowed(i, j)` and nothing else, so it consumes the same concept and adds no third representation of the costs.

`LazyCostMatrix` also bakes in the transformations that the dense path applies in a separate preparation pass, namely mapping forbidden entries to a large sentinel and negating costs when maximising. Doing this at construction is what allows the untouched solver body to work on either type.

10.3 Dependency footprint

`couplr` declares `Rcpp`, `tibble`, `dplyr`, `rlang`, `purrr` and `methods` in `Imports`, and `Rcpp` alone in `LinkingTo`. The recursive closure of hard dependencies is 17 non-base packages, so a default installation brings 18 packages including `couplr` itself. The comparable figures on the same day were 18 for `optmatch`, 8 for `MatchIt` and 7 for `designmatch`.

Two choices sit behind that number. First, we use no external solver: all nineteen algorithms are package source, so there is no dependency on an LP or MIP library and no system solver to install. That choice also keeps `LinkingTo` to `Rcpp` alone, which matters because every entry there forces a compile of that package before `couplr` can be built from source; `couplr` declares only what its headers actually include. Second, everything that serves a single feature lives in `Suggests` and is loaded at call time. The animation and image-morphing functions, the plotting helpers, the interoperability layer for `MatchIt`, `optmatch` and `cobalt`, and the parallel back end are all optional in this sense, so a user who only wants to match pays for none of them.